

AEGIS: A Privacy-Minimised, Browser-Native System for Exam Integrity Monitoring

Vijeth Prashanth Hegde, Raushan Nerkar, Tejas Pantharapalya Venkatesh,
Tejas Tholakalabavi Dinesh, Lakshmi Kiran C M, Jeevika Bangalore Thanuj Kumar
School of Computer Science, University College Dublin, Dublin, Ireland

Abstract—Generative AI lets a candidate paste an examination question into a chatbot and receive a plausible answer in seconds, a threat existing webcam- and lockdown-based proctoring addresses only by trading candidate privacy for partial coverage. We present AEGIS, a browser-native examination integrity system that combines six privacy-minimised behavioural signals, namely tab switching, paste events, keystroke rhythm, response latency, answer-timing consistency, and viewport resizing, into a single interpretable confidence score for human review, with no webcam, microphone, or key-content capture. A non-blocking telemetry architecture guarantees monitoring failure never delays or loses an answer submission. In a ten-session controlled pilot (five honest, five AI-assisted), the scorer separated the two groups with precision, recall, and AUC-ROC of 1.00 and zero false positives, though separation required two signals to co-occur rather than any one alone. A 100-candidate concurrency test sustained 100% WebSocket connection success at 173 MiB memory, with a single-worker latency tail and a small answer-persistence gap under peak write contention addressed through horizontal scaling and acknowledgement-based writes. A seven-responder usability study of the examiner console produced a mean System Usability Scale of 92.86, with a 95% confidence interval clearing the 80-point “excellent” threshold.

Index Terms—academic integrity, exam proctoring, browser telemetry, keystroke dynamics, privacy by design, data minimisation

I. INTRODUCTION

Generative AI has changed the economics of academic dishonesty. Where a student once had to plan to cheat, for example by memorising smuggled answers or recruiting an accomplice, they can now paste an examination question into a chatbot and receive a fluent, original-looking answer within seconds. Conventional plagiarism detection is structurally blind to this: the generated text has no textual overlap with any source document, so a similarity check returns a clean result. Detection must therefore shift from *what* a candidate wrote to *how* they came to write it, that is, the behavioural trace left around the answer rather than the answer itself. Recent work confirms that keystroke timing carries enough signal to separate text that was composed in place from text that was transcribed or pasted from elsewhere [1], [2].

The commercial response has been the *proctoring suite*: webcam monitoring, screen recording, browser lockdown, and in some products biometric face verification. These systems raise the cost of cheating by relocating the examination into the student’s private space. A scoping review of student experience reports recurring anxiety, technical exclusion, and equity concerns, particularly for candidates without a private

room [3]. An ethical analysis identifies surveillance asymmetry and algorithmic misclassification as first-order harms, not side effects [4], and capturing video, audio, and biometric identifiers to invigilate an examination sits uncomfortably with the data-minimisation principle of the General Data Protection Regulation [5]. A lockdown browser is additionally powerless against the cheapest contemporary attack, a second device held outside the camera’s field of view.

AEGIS (Adaptive Exam Guardian and Integrity System) starts from the opposite premise: instrument the browser, not the student. AEGIS observes only the timing and shape of a candidate’s interaction with the examination page: when focus leaves the tab, when a paste occurs and how many characters it carried, the intervals between keystrokes, how long each question took, and whether the viewport was resized. It never requests webcam or microphone permission, never captures the screen, never records which keys were pressed, and never reads the contents of the clipboard. From these signals it computes a single interpretable confidence value in $[0, 1]$, surfaced to the examiner alongside the evidence that produced it. Crucially, AEGIS issues no verdicts: a high score routes a session to a human for review, and the decision to escalate remains an academic judgement, not an automated one.

This report describes the design, implementation, and evaluation of AEGIS, built over one trimester by a six-person team for COMP47250 at University College Dublin in partnership with Microsoft.¹ Our contributions are as follows. First, a privacy-minimised browser telemetry design yielding usable integrity signals from event metadata alone, with the architectural invariant that makes it safe to deploy in a graded assessment: telemetry and answer submission are fully independent, so no monitoring failure can cost a student their work. Second, a six-signal weighted confidence scorer with a per-student keystroke baseline, exposed to examiners as three named sensitivity presets and a per-signal breakdown rather than an opaque verdict. Third, a four-perspective evaluation covering detection accuracy on a controlled pilot, scalability under 100 concurrent sessions, examiner usability via the System Usability Scale, and an analysis of signal validity and parameter choice, reported with its limitations stated explicitly.

¹Source code: <https://github.com/comp47250-team-project/aegis>

II. USER STORIES AND SCENARIOS

AEGIS serves three distinct roles, each with a different relationship to the same evidence. We describe each as a scenario and state the technical challenge it imposed.

A. Aoife, a candidate who wants to be trusted

Aoife sits a 30-minute online examination from her shared flat. Under a webcam-based proctoring tool she would have to point a camera at her bedroom and accept that a recording of her private space is retained by a third party. Her concern is not that she intends to cheat: it is that she cannot prove she did not without surrendering her privacy.

In AEGIS, Aoife sees an explicit consent screen before any monitoring begins, enumerating the four signal categories collected and stating plainly that there is no webcam, microphone, screen recording, or clipboard capture. Only after she consents is any examination content served. During the examination her answers autosave, a countdown shows remaining time, and if she leaves the tab, a banner honestly tells her the event has been recorded. The capability that did not previously exist is sitting a monitored examination with zero biometric capture, while knowing in advance exactly what is observed.

Technical challenge: designing signals that are individually weak and deniable, yet jointly informative, without ever capturing content. This constrains every downstream decision: the paste signal may record a character count but not the characters, and the keystroke signal may record intervals but not keys. It also forces the consent gate onto the server, since a client-side gate can be bypassed by navigating directly to an examination link.

B. Dr. Byrne, an examiner who cannot watch 100 video feeds

Dr. Byrne invigilates a 100-student cohort. Even with video for every candidate she could not usefully watch it; proctoring products acknowledge this by shipping automated flagging, which returns her to the problem of trusting an opaque classifier. What she needs is triage: a defensible ordering of which sessions merit her attention, and enough evidence to judge each one herself.

The AEGIS console gives her a live view of the cohort ranked by risk, with a per-signal breakdown and a post-examination integrity report per candidate. She grades short answers manually, and results are withheld from students until every short answer is graded, so a partially marked paper never reaches a candidate. She can export scores and inspect a paginated event timeline for any session she wants to interrogate. The capability that did not previously exist is evidence-backed triage of a large cohort without reviewing any recording, and without the system making the misconduct decision on her behalf.

Technical challenge: real-time fan-out at cohort scale. Risk summaries must reach the console continuously while up to 100 candidate connections stream events, and the aggregation must not block ingestion. It also forces a presentation problem: a provisional live score from partial data must be visibly

distinguished from the final score at examination close, or an examiner will act on a number that is still moving.

C. An administrator who needs provenance

Institutional deployment raises questions no examiner can answer: who created this account, who opened this examination, and who was flagged and when. AEGIS provides a super-administrator console with user management, an examination inventory, and an append-only audit log recording registrations, open and close events, and flagging decisions. The capability that did not previously exist is reconstructing the provenance of an integrity decision after the fact, a precondition for any appeal.

Technical challenge: recording an audit trail across heterogeneous entity types without coupling it to any one of them, and enforcing role separation strictly enough that an examiner cannot reach another examiner's data.

D. Conflicting requirements

These three roles do not want the same thing: Aoife wants minimal observation, Dr. Byrne wants sufficient evidence, and the institution wants a durable record. AEGIS holds this tension by collecting the least data that still supports human judgement, keeping a human in the decision loop so that the weakness of any individual signal is visible and reasoned about, not hidden.

III. STATE OF THE ART

A. Commercial proctoring suites

The dominant approach to securing remote examinations combines webcam monitoring, screen recording, and browser lockdown, sometimes extended with gaze-tracking. Its strength is coverage: observing the candidate directly detects behaviours no page-level instrument can see, such as a person entering the room.

The critique is now well evidenced. Coghlan et al. [4] argue that these systems create a surveillance asymmetry that is difficult to justify proportionately, and that automated flagging shifts the burden of proof onto the accused. Marano et al.'s scoping review [3] finds consistent reports of candidate anxiety and unequal impact on students lacking private space or stable connectivity, and that acceptability depends less on the monitoring itself than on whether the information flow matches norms candidates already expect of an examination hall, norms this category routinely violates. AEGIS adopts that framing: page-level interaction metadata is closer to what an invigilator legitimately observes than a video feed of a candidate's bedroom is. Two further limitations matter technically: lockdown cannot observe a second device, the cheapest route to a chatbot, and biometric verification degrades unevenly across skin tones, so false accusations are not uniformly distributed. AEGIS deliberately forgoes this entire category; the cost is that it cannot detect an off-screen accomplice, stated as a limitation rather than a solved problem.

B. Keystroke dynamics

Keystroke timing has a long pedigree as a behavioural biometric. Monroe and Rubin [6] established that inter-keystroke latencies identify individuals. Kochegurova and Zateev [7] apply the same family of signals to examinations specifically and show that monitoring can run without the candidate being aware of it, which is precisely why AEGIS treats an explicit consent gate as a design requirement rather than a courtesy. Dhakal et al. [8] provide the large-sample distributional grounding we rely on directly, namely 136 million keystrokes from 168,000 participants, with a median inter-keystroke interval near 175 ms for proficient typists.

The critique relevant to us is that this literature optimises for *identification* under cooperative conditions, with per-user enrolment data gathered in advance. An examination offers neither: there is no enrolment phase, and the subject may be adversarial. Worse, a population-level threshold systematically penalises atypical typists, including the fast, the slow, and users of assistive input or non-Latin input methods. This critique directly shaped our design: we compute a *per-student* baseline from the opening window of each examination rather than comparing against a population constant, and we cap the score contribution when a candidate types *faster* than their own baseline, since speed alone is not evidence of misconduct. Our results bear out the concern: the keystroke signal was not the discriminating factor in our pilot.

C. Behavioural signals and AI-generated text

Closest to our objective is recent work on detecting non-original text from process data. Deane et al. [1] show that keystroke logs distinguish transcribed from composed writing, and Jaha et al. [2] extend this explicitly to AI-assisted cheating. These confirm our premise, but each reports on research datasets gathered from cooperative participants rather than a deployed examination system, and neither addresses the operational question we had to solve, collecting such signals in a real browser without capturing content, and handling collection failure mid-examination. Analysing the submitted answer text itself is an alternative rejected on data-minimisation grounds, since it requires reading candidate work with a generative model. Soltane and Laouar [9] present an integrated cheating-detection system closer to ours in ambition, but rely on camera input.

D. Reasonable alternatives we considered and rejected

Two alternatives deserve explicit treatment. Machine learning from the outset was rejected because no labelled corpus of AI-assisted examination sessions existed at project start, and a transparent weighted rule lets an examiner see which signals drove a score, which a black-box model does not. Hard browser lockdown, disabling copy and paste, blocking tab changes, and forcing fullscreen, was rejected because it converts an evidence-gathering system into an enforcement one, degrading accessibility while remaining trivially defeated by a second device.

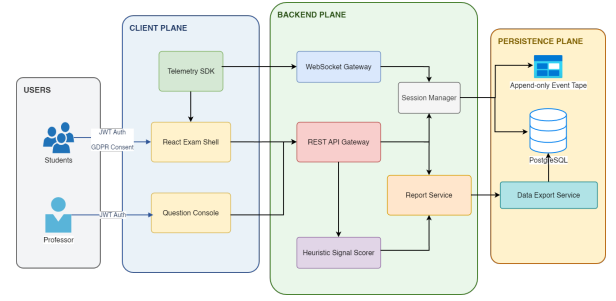


Fig. 1. AEGIS three-plane architecture: client, backend, and persistence. Candidate telemetry and answer submission traverse independent transports, which is what makes the non-blocking guarantee of Sec. IV-D structural rather than defensive.

E. Methods we adopt

For usability measurement we use the System Usability Scale [10], validated at small sample sizes with established interpretive norms [11] and a usability/learnability subscale decomposition [12]. Engineering practice follows continuous-delivery and test-first discipline.

IV. SYSTEM DESIGN AND IMPLEMENTATION

A. Three-plane architecture

AEGIS is organised into three planes (Fig. 1). The client plane comprises a React student examination shell, a browser telemetry SDK, and a separate examiner console. The backend plane is a single FastAPI application exposing a REST API across 13 routers and two WebSocket gateways, one for candidate telemetry and one for live risk summaries, alongside the signal scorer. The persistence plane holds PostgreSQL for relational examination data, Azure Service Bus for asynchronous event dispatch, and Azure Blob Storage for append-only per-session event tapes.

FastAPI was selected over Django and Flask: the integrity scorer is statistical, making Python the natural implementation language, and FastAPI's native asynchronous request model sustains many concurrent WebSocket connections without the retrofitted async layers that Django Channels or an eventlet-based Flask deployment require. Schema validation and a generated API specification materially reduced integration defects across a six-person team.

The frontend is React 18 with TypeScript, built with Vite and styled with Tailwind, suiting a highly stateful examination interface in which the consent gate, countdown and auto-submit, answer autosave, and telemetry instrumentation all mutate independently, while TypeScript enforces the client and server contract at compile time. Vue was rejected on team-familiarity grounds: a two to three week ramp-up inside a twelve-week project was not a defensible cost.

B. Data model and data sources

The schema comprises 17 relational models evolved through 16 schema migrations (Fig. 2). PostgreSQL is the datastore: the domain is strongly relational, spanning telemetry events,

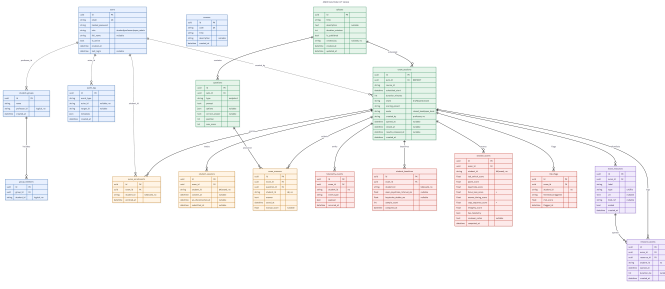


Fig. 2. Entity-relationship diagram. The examination session table is the hub: nine tables reference it with cascading deletion, so removing an examination removes all derived telemetry, baselines, and scores.

student sessions, examination sessions, and quizzes, and answer submission requires strong consistency guarantees, so a document store for telemetry would have doubled operational surface for no analytical gain, since the dominant query is per-student-per-examination rather than a time-range scan. Variable-shape telemetry payloads are held in flexible JSON columns, giving schema flexibility inside the relational engine.

Our data sources are generated rather than collected: the system’s inputs are the behavioural event streams produced by candidates in live examinations, plus examination content authored by examiners. One schema property is a deliberate compromise, not an oversight: candidate identity is a plain identifier string with no foreign-key relationship to the user account table, letting telemetry and scoring be developed and tested against synthetic identifiers independent of the identity schema, at the cost of forgoing referential integrity there, a trade the team would not repeat, since the isolation benefit proved smaller than the ongoing cost of enforcing consistency in application code.

C. Browser telemetry SDK

The SDK emits nine event types: tab blur, tab return, paste, keystroke interval, viewport resize, first keypress, per-question dwell time, resource access, and fullscreen exit. Three privacy properties are enforced in code, not by policy. The paste handler reads clipboard data only to compute a character count and classify the paste as internal or external; the pasted text never enters a payload, and an internal copy-paste emits nothing. The keystroke handler reads modifier flags only to filter shortcuts and emits an interval in milliseconds, so key identity is never transmitted. There is no call to any camera, microphone, or screen-capture browser API anywhere in the frontend source, mechanically verifiable rather than a claim of intent. The signals rest on three standard browser APIs: the Page Visibility API, the Clipboard API, and keyboard event timing.

Events enter a bounded buffer of the most recent 500 events; when the connection is open they are sent immediately, and otherwise accumulate and replay in order on reconnection. Reconnection backs off exponentially from one second to a thirty second ceiling, with three failure conditions that stop retrying permanently: unauthorised, forbidden, and

examination-closed, so a client whose examination has ended does not reconnect indefinitely.

Collection is gated by informed consent, enforced on the server rather than trusted to client. A candidate’s session becomes eligible for telemetry only after an explicit consent action, and the content endpoint stays blocked until then, so navigating directly to an examination link cannot bypass the consent screen the way a client-only check could. This satisfies the data minimisation principle of the General Data Protection Regulation [5] and rests on legitimate interest, conditioned on the collection being metadata-only, informed, and consented to.

D. The non-blocking telemetry invariant

AEGIS enforces one invariant above all others: a telemetry failure must never cause answer loss or block the examination interface. This is structural, not defensive: answers travel over REST and commit directly to the database, telemetry travels over a WebSocket connection, and the two paths share no code. Every telemetry call in the submission routine is non-blocking and never awaited, so an unestablished connection is a no-op rather than an error. The final answer submission is itself wrapped so navigation away proceeds even if that request fails, and the countdown is rendered outside the error boundary that guards question rendering, so a crash while rendering a question cannot stop the clock or prevent auto-submission. On ingestion, database persistence, message-broker publish, and archival append are three independently guarded best-effort steps, and broker publishing is fire-and-forget, so broker latency never stalls the receive loop.

E. Six-signal scoring model

Each session receives a score in the range zero to one, computed as a weighted sum of six independently normalised signal scores:²

$$\begin{aligned} risk = & w_1 \cdot tabswitch + w_2 \cdot paste \\ & + w_3 \cdot keystroke + w_4 \cdot firstkey \\ & + w_5 \cdot timing + w_6 \cdot resize \end{aligned} \quad (1)$$

Weights are supplied by one of three presets (Table I), each summing to 1.0. Presets rather than continuous sliders were chosen so that sensitivity is configurable without exposing raw weights through any interface, which would let a candidate reverse-engineer the threshold.

The keystroke signal is the one component using inferential statistics. A per-student baseline is established from the first five minutes of typing, requiring at least 50 intervals, summarised by its median interval and standard deviation, chosen over the mean because keystroke distributions are heavily right-skewed by pauses for thought. Each subsequent two-minute window with at least ten keystrokes is scored by the standardised deviation of its median against that baseline, and the session takes the maximum across windows. A window

²Two internal database columns retain an earlier naming scheme from an initial design and were not renamed when their meaning changed. We note this here rather than let it go unexplained.

TABLE I
THE SIX BEHAVIOURAL SIGNALS AND THEIR PRESET WEIGHTS. NO SINGLE SIGNAL CAN CROSS THE FLAG THRESHOLD ALONE.

Signal	Basis	Len.	Std.	Str.
tab switch	blur count, short-blur weighted	0.15	0.30	0.35
paste	per-question count and size	0.20	0.25	0.30
keystroke	standardised deviation vs. own baseline	0.25	0.20	0.20
first keypress	time to first keystroke	0.15	0.10	0.07
answer timing	coefficient of variation	0.20	0.10	0.05
resize	viewport-width change	0.05	0.05	0.03

whose median falls at least three standard deviations below the baseline is capped at a low score, so typing faster than one’s own norm is not presumed dishonest, and because the baseline is per-student, atypical typists are compared only against themselves.

A rule-based scorer was chosen over a learned classifier because no labelled corpus of AI-assisted examinations existed at project start, and an examiner reviewing a flagged session must see which signals produced it. Two event types, resource access and fullscreen exit, are recorded as evidence but excluded from the weighted sum, since neither is misconduct alone.

In deployed code a session is flagged, and surfaced as such on the examiner dashboard and in the exported report, once its score reaches 0.40. A separate, higher cutoff of 0.70 gates only the writing of an internal audit record, and is not what a professor sees as flagged. The 0.40 cutoff became the permanent operating threshold in the same change that produced the pilot in Sec. VI-A; both numbers, and the reasoning behind having two, are examined in Sec. VI-D.

F. Deployment, infrastructure, and delivery

Production runs on Azure Container Apps, chosen over Azure Kubernetes Service for managed autoscaling, ingress, and TLS without operating a Kubernetes control plane, a defensible trade for a six-person team on a fixed credit budget. Backend and frontend deploy as independent container applications in front of a managed PostgreSQL server, a message-broker namespace, and a storage account. Infrastructure is declared as reusable code; secrets live in a managed vault, read through a system-assigned identity, never committed or baked into an image. Images are tagged by commit for traceability and rollback, and schema migrations run at container start so schema and code deploy atomically. Local development and continuous integration reproduce the whole stack, so Azure was introduced only once the system already worked locally.

Authentication uses short-lived signed access tokens held in memory, never in browser storage, removing the standard cross-site-scripting exfiltration path, with refresh tokens in a cookie inaccessible to page scripts; passwords are salted and hashed, with a dummy comparison run on unknown accounts to avoid timing-based enumeration. A shared authorisation check over the three roles is applied 44 times across the routers. Delivery is automated: every pull request is linted, type checked, and tested for both backend and frontend,

plus an end-to-end suite against the full stack, and every merge builds, pushes, deploys, and smoke tests the live health endpoint. Later in the project the team additionally stood up an exploratory Kubernetes deployment with a custom domain and automated certificate issuance, not a reversal of the Container Apps choice, but a probe of the multi-replica limitation discussed in Sec. VII.

G. AI Tools in the Development Process

The team used three generative AI assistants throughout development. One was used for architectural exploration, code scaffolding, debugging, and first drafts of documentation; an in-editor completion tool was used for boilerplate whose pattern a human had already established, such as routers, data models, schemas, and test fixtures, and for drafting infrastructure templates and workflow files; and a conversational assistant was used for technology comparison and debugging discussion. Sprint bookkeeping was partly automated through an integration with the project tracker.

Our governing rule was that no AI-generated code was merged unless the responsible team member could explain every line of it, enforceable rather than aspirational through three mechanisms. Every change reached the main branch by pull request requiring at least one peer approval and passing continuous integration, so a second member had to read and accept the code. Human-authored tests remained the specification of correctness: where AI proposed scorer logic, the committed unit tests, not the generated implementation, defined expected behaviour, and the scoring components carry 77 dedicated tests. The live interview also obliged each member to defend any part of the codebase, making unexplained code a direct personal risk.

Certain work was deliberately not delegated. The choice of architecture and technology stack, the design of the evaluation protocol, the interpretation of results, and the analysis in Secs. VI and VII are the team’s own. AI assistance materially accelerated scaffolding and infrastructure work; it was least useful where the problem was genuinely novel, and the team substituted its own reasoning from the literature.

V. SOFTWARE ITERATIONS REVIEW

Development ran from 1 June to 7 August 2026, producing 109 non-merge commits across 27 reviewed pull requests and closing 118 of 126 tracked issues, spanning 14 of 15 epics;

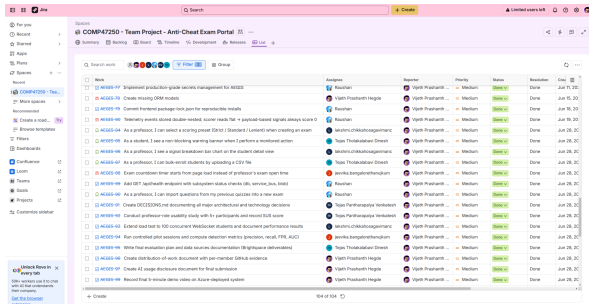


Fig. 3. Jira board at project close: 126 of 126 tracked issues completed, spanning 15 of 15 epics.

the remaining work covers evaluation closure and final submission. All six members committed throughout the trimester rather than in bursts at the deadlines; weekly commit volume peaked in the weeks covering the scoring engine and the evaluation campaign respectively. Every change reached the main branch through a pull request gated on peer approval and passing continuous integration (Fig. 3).

A. Iteration 1: foundations

The first iteration deliberately built no integrity features. The team stood up the local development stack, the relational model layer, schema migrations, authentication, and role-based access control. This ordering was itself a response to mentor feedback received before implementation began: the team had proposed introducing Azure early, and was advised that cloud services add overhead in setup, debugging, and testing, and that simpler alternatives should come first. The team adopted a container-based local stack for development and continuous integration, introducing Azure only once the system worked locally, which proved correct, since it kept the entire stack runnable on a laptop for the whole project.

B. Iteration 2: the examination shell

The team built the candidate-facing shell: consent gate, question rendering, answer autosave, countdown with auto-submit, and the first telemetry signal, tab blur and return. The live examiner monitor followed in the same iteration, which turned out to matter: seeing real events arrive in a console immediately exposed that the event payloads were too sparse to interpret, feedback the team acted on two iterations later.

C. Iteration 3: the scoring engine

Signals were implemented across two pull requests, tab-blur and paste scorers followed by answer-timing and resize scorers, then the preset system. The most consequential decision of the project was taken here. The original proposal centred on a machine-learning classifier; a mentor observed that machine learning seemed like overkill for this use case, especially given the project timeline and the lack of labelled data, and recommended treating it as an extension rather than a core module. The team accepted this and implemented a transparent weighted rule instead. In hindsight this was the

difference between shipping a working scorer and shipping a classifier trained on the team’s own synthetic assumptions.

Two further mentor concerns from the same period shaped the design permanently. A warning that keylogging and clipboard scraping would raise questions of consent and data storage under the General Data Protection Regulation led directly to the decision to capture no webcam or microphone, to record keystroke intervals but never key values, and to gate collection behind server-verified consent. A recommendation to run a quick feasibility check on every detection mechanism, because browser applications carry more restrictions than a standalone local application, produced an early feasibility spike, which is why the signal set contains only events the team had already confirmed fire reliably.

D. Iteration 4: responding to examiner-driven feedback

This iteration was almost entirely externally directed. Mentors asked why the team chose a relational database over a document store, and whether it was reading from a shared cache, posing a scenario in which an examiner authors a quiz in one region while candidates read it from another. The team analysed the access pattern and added a per-process quiz cache, having found repeated identical reads on every question fetch. Further requests in the same sessions produced candidate grouping into cohorts, a richer event timeline addressing the observation that the existing one showed too little detail, a super-administrator console with an audit log a mentor proposed instead of surfacing provenance directly to examiners, an explicit answer-save affordance, and a fix ensuring an examination closes for every candidate once an examiner ends it. One subtle correction also came from this feedback: copying and pasting within the examination window was being reported when it should not have been flagged, which is why internal pastes are now classified on the client and emit no event.

E. Iteration 5: turning claims into evidence

This iteration produced the empirical backbone of the evaluation. The team designed and ran a controlled detection pilot with labelled honest and AI-assisted cohorts, extended load testing from 50 to 100 concurrent candidates with answer submission included in the workload, and ran the examiner usability study. Together these three activities turned the detection accuracy claim, which had until then rested only on the scorer’s design, into a measured result with its limitations stated.

F. Iteration 6: hardening and closure

The final iteration delivered open-book examinations with examiner-curated resource allow-lists, a feature that emerged from a mentor asking whether the team had considered an in-application browser, plus fullscreen-exit detection and a fix for an examiner live-view desynchronisation after automatic closure. The team also stood up the exploratory Kubernetes deployment.

G. What we would do differently

The scoring component could have been improved with machine learning integration. Also, load testing could be conducted with more replicas and number of student sessions (50,000 or more).

VI. EVALUATION AND RESULTS

We evaluate AEGIS from four independent perspectives: whether it detects the behaviour it targets, whether it scales, whether examiners can use it, and whether its parameters are defensible. Reporting all four, rather than only the most favourable, reflects the priority the team placed on this section following its own review of progress.

A. Detection accuracy

The team ran ten controlled examination sessions against the live scoring pipeline under the standard preset: five honest sessions (answers typed from scratch, no tab-switching, no pasting) and five simulated AI-assisted sessions (three or more tab switches, and a paste exceeding 100 characters on at least one question). Ground truth was assigned before sessions ran and was not shared with the scorer. Results are evaluated at the deployed flagging threshold of 0.40, the permanent operating point used by the examiner dashboard and the exported report, distinct from a separate higher internal threshold that only governs when an audit record is written.

All ten sessions were classified correctly. Precision, recall, and AUC-ROC were each 1.00 against targets of at least 0.80, 0.75, and 0.80, with a false positive rate of 0.00 against a target of at most 0.10. Honest scores ranged 0.106 to 0.175 and assisted scores 0.438 to 0.641, so every assisted score exceeds every honest score and the AUC-ROC of 1.00 reflects exact rank separation rather than curve-fitting, with no score near the boundary. A perfect score on ten sessions invites scrutiny rather than settling the question, so the team examined the mechanism. Separation was driven by co-occurrence, not by any single signal: tab-switch and paste carry the two largest weights under the standard preset, so even a maximal single-signal score cannot reach 0.40 without a second contributing signal. Every simulated session combined tab-switching and pasting by construction, so the pilot demonstrates that the scorer correctly aggregates a compound behaviour; it does not demonstrate that a single-signal case would be caught. We regard this as the honest headline finding.

B. Scalability

The telemetry gateway was load-tested with a single back-end worker, at 50 and 100 concurrent candidate connections (Scenarios A and B), each run three times for stability, with one candidate answer submitted per session at the midpoint of a two-minute steady state.

Three metrics met their target cleanly at both scales: connection success, backend memory, and zero server errors (Table II). The other two differ by scale, so we report each rather than a single count. Answer latency met its target at 50 candidates but not at 100, where all candidates submit

TABLE II

LOAD TEST RESULTS AGAINST TARGETS. LATENCY IS THE 99TH PERCENTILE ACROSS THREE RUNS; THE MEDIAN STAYED AT 545 MS AT 100 CANDIDATES WITH ZERO REQUEST FAILURES. PERSISTENCE IS THE FRACTION OF ANSWERS RETURNING A SUCCESS RESPONSE THAT WERE THEN FOUND IN THE DATABASE.

Metric	Target	50	100
Connection success	100%	100%	100%
Answer latency	≤ 800 ms	390–545 ms	686–952 ms
Backend memory	< 800 MB	172 MiB	173 MiB
Server errors	0	0	0
Answers persisted	100%	96%	99%

within roughly the same one-second window and the burst lands on a single worker; the 545 ms median and zero request failures indicate a tail effect rather than systemic saturation, and horizontal scaling removes this bottleneck.

C. Examiner usability

Seven participants completed a moderated, task-based session with the examiner console, followed by the ten-item System Usability Scale [10]. Scores were computed from the raw per-item responses rather than a pre-aggregated figure, giving a mean of 92.86 (median 92.5, sample standard deviation 4.66, range 85 to 100). At a sample size of seven a t -interval is the appropriate formal treatment: the 95% confidence interval is 88.6 to 97.2, entirely above both the population mean of 68 and the 80-point excellent threshold established by Bangor et al. [11] because the interval, not only the point estimate, clears both bars. By subscale [12] the console scores 94.20 for usability and 87.50 for learnability, so it is judged easy to operate and easy to learn to comparable degrees.

The finding's limits carry the same weight as its result: the instrument captured the ten standard items only. Task success, time-on-task, and qualitative debrief data were not recorded for this cohort, so the team can report that participants rated the console highly, but not precisely where in the interface they struggled; a follow-up session is planned that captures task-level metrics.

D. Signal validity and parameterisation

Two parameters merit direct justification. The keystroke signal's 400 ms reference point is not a measured population mean but a deliberate upper bound: Dhakal et al. report a median inter-keystroke interval near 175 ms for proficient typists [8], and setting the reference above that median biases the signal toward under-flagging fast typists at the cost of a less sensitive detector, a trade made deliberately in the privacy-preserving direction. Setting it at the measured median instead would flag roughly half of all honest fast typists as anomalous by construction.

The two thresholds described in Sec. IV-E, 0.40 for the professor-facing flag and 0.70 for the internal audit record, represent a precision-recall trade rather than an inconsistency. At 0.70 a session must combine three contributing signals, minimising false positives at the cost of requiring stronger

combined evidence; at 0.40 the two-signal combination of Sec. VI-A suffices. The pilot cannot determine which point is optimal in production, since a ten-session sample with clean separation does not exercise the boundary between them; refining it with a larger sample is planned future work.

VII. DISCUSSION

Five points bound what we can claim, and each has a planned next step.

Ten sessions demonstrate that the scorer aggregates a compound signal correctly; they do not establish population-level precision or recall, and the single-signal case, tab-switching without pasting or the reverse, was not tested in this round. A larger, externally recruited sample exercising the decision boundary is the natural next step.

Seven participants is small, and the confidence interval in Sec. VI-C reflects that; task-level success and timing data were not recorded in this round, so the team knows the console was rated highly but not precisely why. A follow-up session capturing task-level metrics is planned to build on this result.

The live risk aggregator, in-memory authentication-token revocation, and the per-process quiz cache (Sec. IV) all hold state in one process's memory. AEGIS is, as deployed, correct for one replica, and the team's next infrastructure step is to move this state to a shared store such as Redis before running multiple replicas, precisely the scaling question the exploratory Kubernetes deployment began to probe.

Sec. VI-B measured a small fraction of answer submissions returning success without persisting, under a synchronised single-worker write burst, a correctness effect in an examination system rather than merely a performance one. The remediation planned first is a server-side acknowledgement the client can verify, backed by horizontal scaling to remove the single-worker constraint. A planned 90-day deletion window for raw telemetry is scheduled for a subsequent iteration.

No behavioural signal system can see a determined evasion, such as a second device outside any browser's observation. AEGIS does not claim otherwise. Its contribution is evidence for a human reviewer, not a verdict, and the team considers that framing a feature of the design rather than a gap to close. On the two choices Sec. III flagged as reasonable alternatives: the pilot's clean separation was achieved with a transparent rule, giving no evidence that a learned classifier was needed for this sample, though it cannot rule that out at population scale; and no evasion in the pilot required defeating a lockdown the team chose not to build, consistent with the view that lockdown defends against the wrong attacker.

Planned future work follows directly from these five points: a logistic regression over the six existing scores, validated by leave-one-examination-out cross-validation now that a labelled pilot exists; a per-signal look at false-positive drivers at the single-signal boundary; shared state in a fast in-memory store for multi-replica correctness; a durable, retrievable scoring worker in place of the current best-effort background task; and an acknowledgement-based write path for answer submission.

VIII. CONCLUSION

AEGIS shows that defensible integrity evidence does not require watching the candidate. Six privacy-minimised behavioural signals, combined into a single interpretable score and routed to a human reviewer rather than an automated verdict, avoid the webcam and lockdown proctoring model entirely while still giving examiners something actionable. Our evaluation supports this claim across four perspectives with its limitations attached rather than omitted: a controlled pilot showed the scorer separates honest from AI-assisted sessions when two or more signals co-occur; the system sustained 100 concurrent candidates with zero server errors, subject to a single-worker latency tail and a small answer-persistence gap the team is addressing through horizontal scaling and acknowledgement-based writes; and examiners rated the console highly on a small sample. AEGIS does not claim to catch every determined evader; it claims to make honest behaviour verifiable and dishonest behaviour reviewable, which is the problem the project set out to address.

REFERENCES

- [1] P. Deane, M. Zhang, J. Hao, and C. Li, "Using keystroke dynamics to detect nonoriginal text," *Journal of Educational Measurement*, vol. 63, no. 1, p. e12431, 2025.
- [2] F. Jaha, A. Kartit, and M. Fertat, "Leveraging keystroke dynamics to detect identity fraud and AI-driven cheating in online education," in *Proc. 2nd Int. Symp. on Generative AI and Education (ISGAIE 2025)*, ser. Lecture Notes on Data Engineering and Communications Technologies. Springer, 2025, pp. 487–498.
- [3] E. Marano, P. M. Newton, Z. Birch, M. Croombs, C. Gilbert, and M. J. Draper, "What is the student experience of remote proctoring? A pragmatic scoping review," *Higher Education Quarterly*, vol. 78, no. 3, pp. 1031–1047, 2024.
- [4] S. Coghlan, T. Miller, and J. Paterson, "Good proctor or 'Big Brother'? Ethics of online exam supervision technologies," *Philosophy & Technology*, vol. 34, no. 4, pp. 1581–1606, 2021.
- [5] European Parliament and Council of the European Union, "Regulation (EU) 2016/679 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data (General Data Protection Regulation)," *Official Journal of the European Union*, vol. L119, pp. 1–88, May 2016.
- [6] F. Monrose and A. D. Rubin, "Keystroke dynamics as a biometric for authentication," *Future Generation Computer Systems*, vol. 16, no. 4, pp. 351–359, Feb. 2000.
- [7] E. A. Kochegurova and R. P. Zateev, "Hidden monitoring based on keystroke dynamics in online examination system," *Programming and Computer Software*, vol. 48, no. 6, pp. 385–398, 2022.
- [8] V. Dhakal, A. M. Feit, P. O. Kristensson, and A. Oulasvirta, "Observations on typing from 136 million keystrokes," in *Proc. 2018 CHI Conf. on Human Factors in Computing Systems*. Montreal, QC, Canada: ACM, Apr. 2018, pp. 1–12.
- [9] M. Soltane and M. R. Laouar, "A smart system to detect cheating in the online exam," in *Proc. Int. Conf. on Information Systems and Advanced Technologies (ICISAT)*. IEEE, 2021, pp. 1–5.
- [10] J. Brooke, "SUS: A 'quick and dirty' usability scale," in *Usability Evaluation in Industry*, P. W. Jordan, B. Thomas, B. A. Weerdmeester, and I. L. McClelland, Eds. London, U.K.: Taylor & Francis, 1996, pp. 189–194.
- [11] A. Bangor, P. T. Kortum, and J. T. Miller, "An empirical evaluation of the System Usability Scale," *International Journal of Human-Computer Interaction*, vol. 24, no. 6, pp. 574–594, 2008.
- [12] J. R. Lewis and J. Sauro, "The factor structure of the System Usability Scale," in *Human Centered Design (HCD 2009)*, ser. Lecture Notes in Computer Science. Berlin, Heidelberg: Springer, 2009, pp. 94–103.